



UNIVERSIDADE FEDERAL DO PIAUÍ
CENTRO DE CIÊNCIAS DA NATUREZA
DEPARTAMENTO DE COMPUTAÇÃO
DISCIPLINA: SISTEMAS DISTRIBUÍDOS
DOCENTE: PROF. DR. JOSÉ RODRIGUES TORRES NETO

FELIPE GABRIEL SOARES RODRIGUES
LUANDERSON OLIVEIRA DE CARVALHO
LUCAS VILARINHO CARVALHO MARTINS CAMARÇO

**DESENVOLVIMENTO DE UM PROCESSADOR DISTRIBUÍDO DE TRANSAÇÕES
EM TEMPO REAL**

Teresina

2026

1. Tecnologias utilizadas:

Para o desenvolvimento dessa aplicação, as seguintes tecnologias foram adotadas:

- Python 3.14: Linguagem de programação escolhida para construção do código-fonte executor das partes de gerador e de processador da aplicação.
- Kafka: Tecnologia integrada ao python utilizada para intermediar a comunicação entre o lado cliente e o lado servidor da aplicação.
- Docker: Ferramenta utilizada para criação de um contêiner, que possui os requisitos instalados previamente, garantindo o funcionamento da aplicação em qualquer máquina.

2. O que foi desenvolvido:

Para esta aplicação, que foi construída com base nos conhecimentos adquiridos de Sistemas Distribuídos, a linguagem Python foi utilizada como base para criar um sistema de processamento de transações bancárias. Este sistema funciona com dois principais atores: O produtor/gerador e o consumidor/processador. O primeiro é responsável por gerar automaticamente, de forma não determinística, transações bancárias realizadas por atores previamente cadastrados. Já o segundo é responsável por receber as transações geradas e, com base no cruzamento das informações de cada transação com um conjunto de regras predefinido, tomar a decisão de bloquear ou aprovar a procedência de cada transação.

Ademais, para que a comunicação entre as partes pudesse ser eficiente, dividimos as funções entre dois nós (um para cada ator) integrados a partir do Kafka, que gerencia todo o processo de encaminhamento de informações entre nós, trabalhando para garantir a tolerância a falhas. Para compactar a aplicação e fazê-la funcionar independente das especificações da máquina do usuário, a ferramenta Docker foi utilizada para containerização, agrupando as bibliotecas e os recursos necessários para garantir o funcionamento.

Em relação ao processo de criação das transações, três cidades foram fixadas, com sua localização precisa definida previamente, sendo estas: Teresina, São Paulo e Rio de Janeiro. Essas localidades tiveram suas coordenadas geográficas escritas para registro e posterior cálculo de distância em relação ao local de compra das transações. Os usuários, que são os supostos responsáveis financeiros pelas compras, também possuem dados de localidade padrão cadastrados, para cruzar referências quando as compras forem feitas. Conforme o tempo passa, essas transações são geradas e, a cada instante, há uma probabilidade de ocorrência de um evento anormal, que deve ser devidamente administrado pela aplicação.

Paralelamente, três regras são utilizadas para classificar a transação em aprovada ou bloqueada, são elas:

1. Alta frequência detectada: Múltiplas transações (mais de 8, por exemplo) em um curto período de tempo, logo seria altamente improvável que fossem transações válidas, por isso são bloqueadas automaticamente.
2. Compra impossível: Período de tempo extremamente curto entre duas transações geograficamente distantes, de modo que a distância entre os dois locais não possa ser percorrida por um avião comercial, logo a transação deve ser bloqueada por ter sido impossível de ser realizada.
3. Compra improvável: Dado um histórico de compras, se o valor da compra de um cliente ultrapassa a soma entre o valor médio de compra e três vezes o valor do desvio padrão de sua distribuição, a transação deve ser bloqueada por possuir 99,7% de chance de ser uma fraude, baseado em métodos estatísticos.

As demais transações são consideradas normais, por questões de simplificação da implementação deste projeto.

3. Como executar a aplicação:

O projeto foi construído para rodar em contêineres, garantindo que o ambiente de execução seja o mesmo independentemente da sua máquina. O sistema orquestra a comunicação entre o broker do Apache Kafka, o gerador de transações e o motor antifraude de forma automática.

Pré-requisitos

- **Windows ou macOS:** É recomendada a instalação do **Docker Desktop**.
- **Linux:** Certifique-se de ter o **Docker Engine** e o plugin **Docker Compose** instalados em sua distribuição.

Execução

1. Inicie o Docker Desktop ou Docker Engine e aguarde até que o serviço conste como em execução.
2. Abra o terminal da sua escolha.
3. Navegue até a pasta raiz do projeto (o diretório que contém o arquivo `docker-compose.yml`).
4. Para construir as imagens da aplicação e iniciar o sistema, execute:

(terminal localizado na raiz do projeto)

```
docker compose up --build
```

5. O Kafka levará alguns instantes para estabilizar. Assim que ele estiver pronto, o gerador de transações e o processador antifraude iniciarão seus processos automaticamente.

Comandos úteis para avaliação e testes

Como a aplicação simula cenários contínuos de transações e retém os dados usando volumes do Docker, separamos alguns comandos úteis para facilitar a avaliação do sistema. Você pode abrir uma nova aba no seu terminal para executá-los:

- **Acompanhar os alertas de fraude (Monitor):** Para visualizar o motor em funcionamento e ver os bloqueios ocorrendo em tempo real, observe os logs específicos do processador:

(terminal localizado na raiz do projeto)

```
docker logs -f app-motor-antifraude
```

- **Parar o sistema (Mantendo o histórico):** Se precisar interromper a simulação de forma segura, mas quiser manter o banco de dados do Kafka intacto para análises futuras:

(terminal localizado na raiz do projeto)

```
docker compose down
```

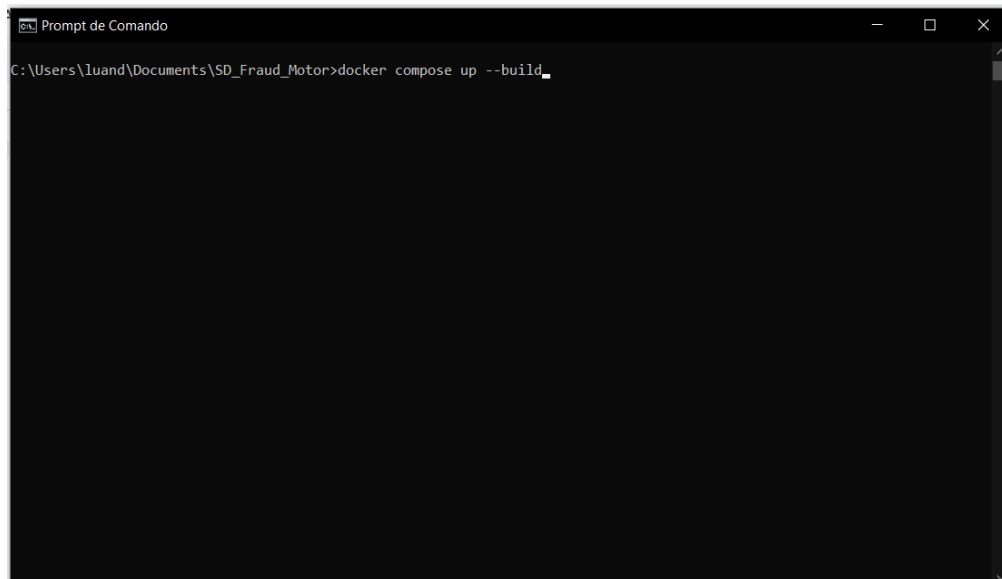
- **Reiniciar a simulação do zero:** Para fazer uma nova apresentação limpa, é necessário apagar os volumes para resetar a "memória" do Kafka:

(terminal localizado na raiz do projeto)

```
docker compose down -v
```

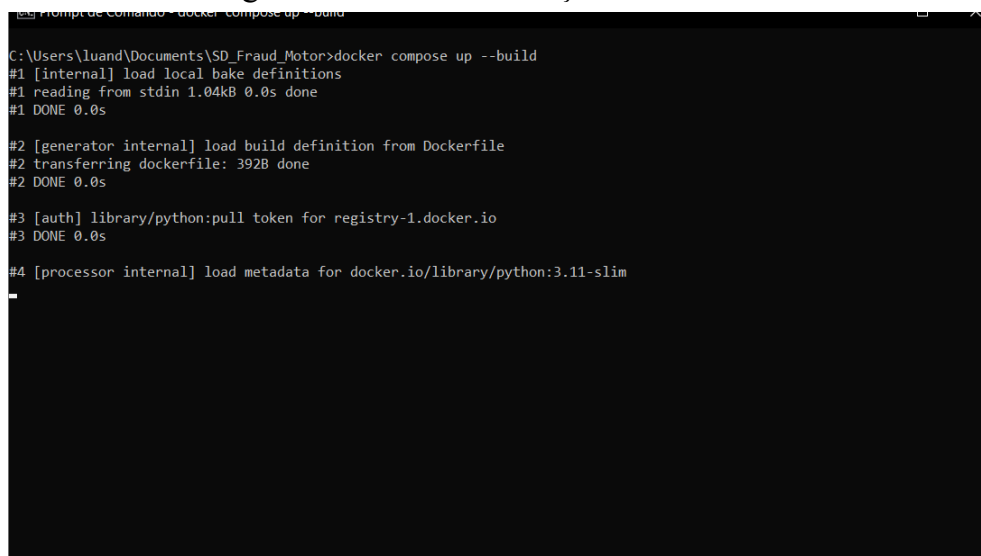
4. Exemplos no Terminal e na interface Docker

Figura 1. Comando de montagem dos nós no terminal



```
Prompt de Comando
C:\Users\luand\Documents\SD_Fraud_Motor>docker compose up --build_
```

Figura 2. Início da execução do comando



```
Prompt de Comando - docker compose up --build
C:\Users\luand\Documents\SD_Fraud_Motor>docker compose up --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.04kB 0.0s done
#1 DONE 0.0s

#2 [generator internal] load build definition from Dockerfile
#2 transferring dockerfile: 392B done
#2 DONE 0.0s

#3 [auth] library/python:pull token for registry-1.docker.io
#3 DONE 0.0s

#4 [processor internal] load metadata for docker.io/library/python:3.11-slim
_
```

Figura 3. Transações processadas no terminal.

```
app-motor-antifraude | [Processando] Usuário: usuario_666 | Valor: R$369.01 | Local: Rio_Janeiro | Offset: 1791
app-motor-antifraude | [Transação Aprovada com Sucesso.
app-motor-antifraude | Resultado: APROVADO
app-motor-antifraude | [Total Envios: 44] Tópico: transacoes_pendentes | Partição: 0 | Offset: 1792

app-motor-antifraude |
app-motor-antifraude | Recebido: {'Id_Trasacao': 'ed805031-5d06-4bfa-81af-5c77f08fad01', 'id_conta': 'usuario_011', 'va
app-motor-antifraude | lor': 99.88, 'localizacao': '-23.5505,-46.6333', 'timestamp': '2026-06-30T21:44:04.745260+00:00', 'offset': 1792}
app-motor-antifraude | [Processando] Usuário: usuario_011 | Valor: R$99.88 | Local: Sao_Paulo | Offset: 1792
app-motor-antifraude | [Transação Aprovada com Sucesso.
app-motor-antifraude | Resultado: APROVADO

app-motor-antifraude |
app-motor-antifraude | [Total Envios: 45] Tópico: transacoes_pendentes | Partição: 0 | Offset: 1793

app-motor-antifraude | Recebido: {'Id_Trasacao': '5d942589-e8d7-4fd7-b317-397c07e3dfc4', 'id_conta': 'usuario_069', 'va
app-motor-antifraude | lor': 481.51, 'localizacao': '-23.5505,-46.6333', 'timestamp': '2026-06-30T21:44:06.848879+00:00', 'offset': 1793}
app-motor-antifraude | [Processando] Usuário: usuario_069 | Valor: R$481.51 | Local: Sao_Paulo | Offset: 1793
app-motor-antifraude | [Transação Aprovada com Sucesso.
app-motor-antifraude | Resultado: APROVADO

View in Docker Desktop View Config Enable Watch Detach
```

Figura 4. Contêineres montados no Docker Desktop

Containers [Give feedback](#)

Container CPU usage 3.19% / 1600% (16 CPUs available) Container memory usage 350.02MB / 5.33GB [Show charts](#)

☐ ☒ Only show running containers

☐	☐	Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory	Actions
☐	☒	sd_fraud_motor	-	-	-	2.94%	350.02MB / 16.3'	6.	
☐	☒	broker-kafka	21763b4adf48	apache/ka	9092:9092	2.56%	310.4MB / 5.46G	5.	
☐	☒	app-motor-antifraude	012dd8e69b8f	sd_fraud_n		0.15%	20.6MB / 5.46GB	0.	
☐	☒	app-gerador	ff2b9d19fd0e	sd_fraud_n		0.23%	19.02MB / 5.46G	0.	

Figura 5. Log de transações na interface Docker

☐	2026-06-30 18:49:21		sd_fraud_motor/generator	◆ [Total Envios: 195] Tópico: transacoes_pendentes Partição: 0 Offset: 2003
☐	2026-06-30 18:49:21		sd_fraud_motor/processor	
☐	2026-06-30 18:49:21		sd_fraud_motor/processor	Recebido: {'Id_Trasacao': '319bc9d8-f5aa-4b9a-abf3-581251a9853e', 'id_conta': 'usuario_...
☐	2026-06-30 18:49:21		sd_fraud_motor/processor	[Processando] Usuário: usuario_666 Valor: R\$299.71 Local: Rio_Janeiro Offset: 2003
☐	2026-06-30 18:49:21		sd_fraud_motor/processor	✅ Transação Aprovada com Sucesso.
☐	2026-06-30 18:49:21		sd_fraud_motor/processor	Resultado: APROVADO
☐	2026-06-30 18:49:23		sd_fraud_motor/generator	◆ [Total Envios: 196] Tópico: transacoes_pendentes Partição: 0 Offset: 2004
☐	2026-06-30 18:49:23		sd_fraud_motor/processor	Recebido: {'Id_Trasacao': '85d16b48-3a9e-44b4-8c86-66bda23fcd57', 'id_conta': 'usuario_...
☐	2026-06-30 18:49:23		sd_fraud_motor/processor	[Processando] Usuário: usuario_000 Valor: R\$119.55 Local: Sao_Paulo Offset: 2004
☐	2026-06-30 18:49:23		sd_fraud_motor/processor	✅ Transação Aprovada com Sucesso.
☐	2026-06-30 18:49:23		sd_fraud_motor/processor	Resultado: APROVADO
☐	2026-06-30 18:49:25		sd_fraud_motor/generator	◆ [Total Envios: 197] Tópico: transacoes_pendentes Partição: 0 Offset: 2005
☐	2026-06-30 18:49:25		sd_fraud_motor/processor	Recebido: {'Id_Trasacao': 'f720efca-601c-481b-bd49-4f349b3ef5b1', 'id_conta': 'usuario_...
☐	2026-06-30 18:49:25		sd_fraud_motor/processor	[Processando] Usuário: usuario_000 Valor: R\$353.8 Local: Sao_Paulo Offset: 2005
☐	2026-06-30 18:49:25		sd_fraud_motor/processor	✅ Transação Aprovada com Sucesso.
☐	2026-06-30 18:49:25		sd_fraud_motor/processor	Resultado: APROVADO

Figura 6. Exemplo de transação bloqueada

☐	2026-06-30 18:52:18		sd_fraud_motor/processor	[Processando] Usuário: usuario_000 Valor: R\$420.49 Local: Sao_Paulo Offset: 2098	▼
☐	2026-06-30 18:52:18		sd_fraud_motor/processor	✅ Transação Aprovada com Sucesso.	▼
☐	2026-06-30 18:52:18		sd_fraud_motor/processor	Resultado: APROVADO	▼
☐	2026-06-30 18:52:20		sd_fraud_motor/generator	Simulando evento de valor anômalo na transação do usuário : usuario_666!	▼
☐	2026-06-30 18:52:20		sd_fraud_motor/generator		▼
☐	2026-06-30 18:52:20		sd_fraud_motor/processor		▼
☐	2026-06-30 18:52:20		sd_fraud_motor/processor	Recebido: {'Id_Trasacao': '6318d501-4ffa-4b4a-985d-96fe9b7cb043', 'id_conta': 'usuario_...	▼
☐	2026-06-30 18:52:20		sd_fraud_motor/processor		▼
☐	2026-06-30 18:52:20		sd_fraud_motor/processor	[Processando] Usuário: usuario_666 Valor: R\$2788.0 Local: Rio_Janeiro Offset: 2099	▼
☐	2026-06-30 18:52:20		sd_fraud_motor/generator	◆ [Total Envios: 280] Tópico: transacoes_pendentes Partição: 0 Offset: 2099	▼
☐	2026-06-30 18:52:20		sd_fraud_motor/processor	🚨 FRAUDE DETECTADA [Outlier Estatístico]!	▼
☐	2026-06-30 18:52:20		sd_fraud_motor/processor	Detalhes: R\$2788.0 fugiu do padrão. Média: R\$267.59 Limite Dinâmico: R\$716.96	▼
☐	2026-06-30 18:52:20		sd_fraud_motor/processor	Resultado: BLOQUEADO_VALOR	▼
☐	2026-06-30 18:52:22		sd_fraud_motor/processor		▼
☐	2026-06-30 18:52:22		sd_fraud_motor/processor	Recebido: {'Id_Trasacao': '0a958398-ca4f-4e22-8506-2e6b052a7f89', 'id_conta': 'usuario_...	▼
☐	2026-06-30 18:52:22		sd_fraud_motor/processor		▼
☐	2026-06-30 18:52:22		sd_fraud_motor/generator	◆ [Total Envios: 281] Tópico: transacoes_pendentes Partição: 0 Offset: 2100	▼
☐	2026-06-30 18:52:22		sd_fraud_motor/processor	[Processando] Usuário: usuario_067 Valor: R\$216.67 Local: Teresina Offset: 2100	▼
☐	2026-06-30 18:52:22		sd_fraud_motor/processor	✅ Transação Aprovada com Sucesso.	▼
☐	2026-06-30 18:52:22		sd_fraud_motor/processor	Resultado: APROVADO	▼